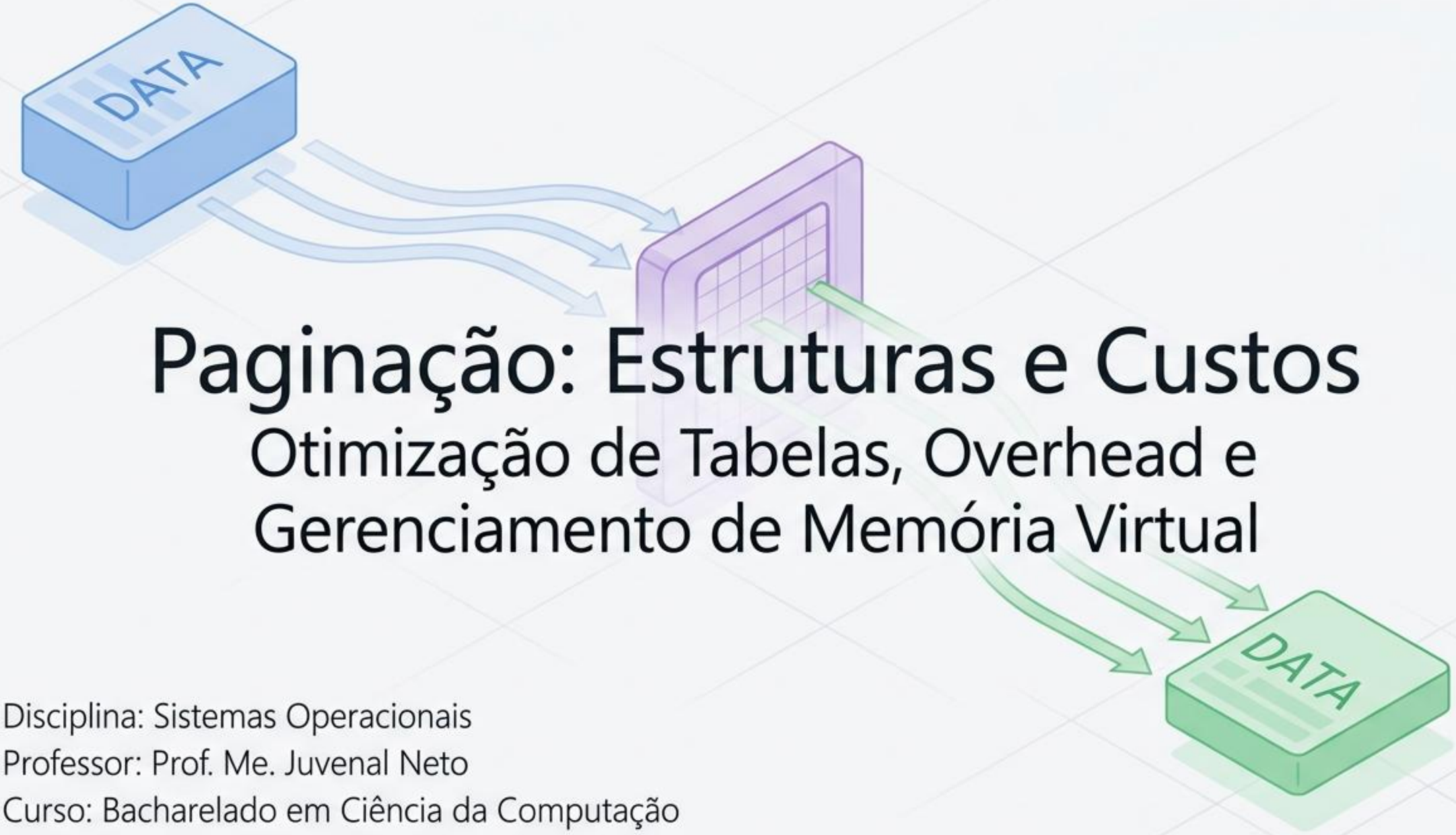
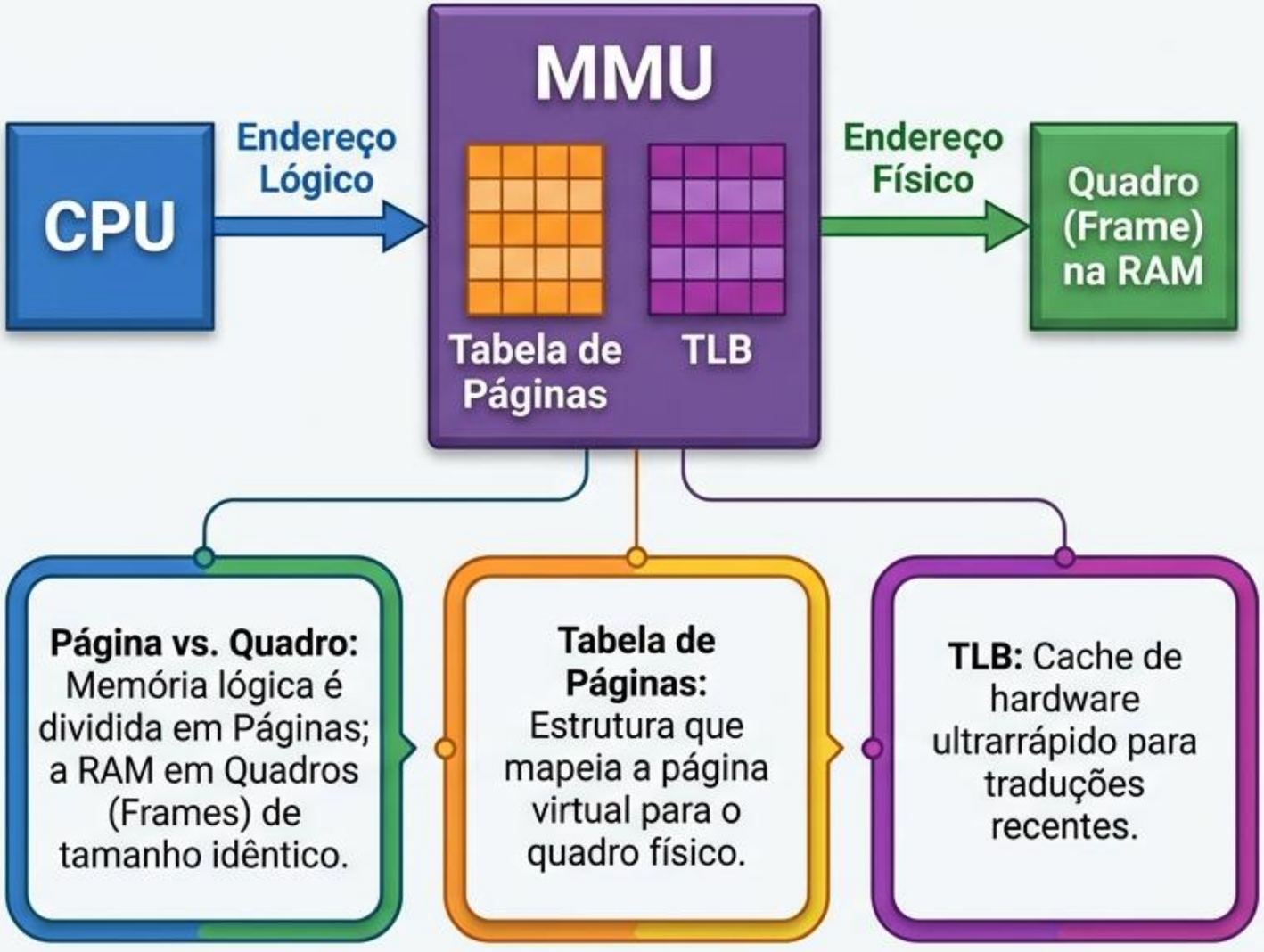


Paginação: Estruturas e Custos

Otimização de Tabelas, Overhead e Gerenciamento de Memória Virtual

Disciplina: Sistemas Operacionais
Professor: Prof. Me. Juvenal Neto
Curso: Bacharelado em Ciência da Computação
Instituição: Universidade do Estado de Mato Grosso (UNEMAT) – Campus de Alto Araguaia

Gemini Notebook



A Questão Central:

Como organizar o mapeamento de espaços virtuais gigantescos sem desperdiçar gigabytes de memória RAM com tabelas vazias e sem tornar a tradução de endereços inaceitavelmente lenta?

Gemini Notebook

A Tabela de Páginas e o Cache TLB: Detalhes e Comparação



Tabela de Páginas

Extreme manificação



Quadro (Frame): Onde o dado reside fisicamente na RAM.

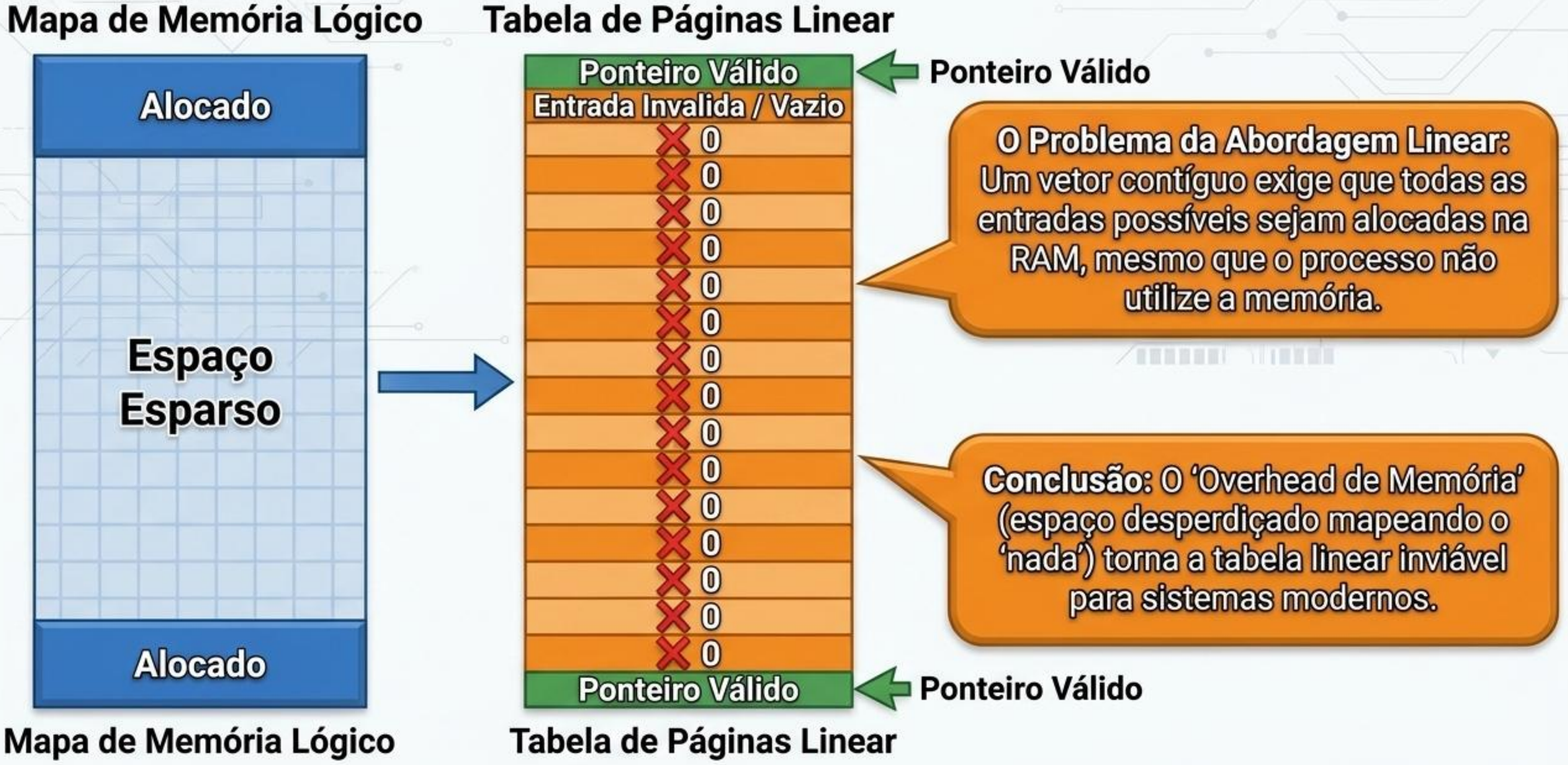
V/I (Presente): 1 = Na RAM; 0 = Page Fault (está no disco).

Permissões: Proteção em hardware para Leitura/Escrita e isolamento Usuário/Sistema.

Tabela de Páginas	TLB
Reside na RAM	Reside na MMU (CPU)
Gerenciada pelo SO	Gerenciada por hardware
Completa (mapeia todo o processo)	Parcial (apenas traduções recentes)
Consulta lenta	Acesso ultrarrápido (~1 ciclo)

Gemini Notebook

A Ineficiência da Abordagem Linear: O Problema do Desperdício Espacial



Visualização do Desperdício de Memória RAM

Gemini Notebook

Custo da Tabela Linear: Prova Matemática

Cenário: Arquitetura de 32 bits, Páginas de 4 KiB, Entradas (PTE) de 4 bytes.

Passo 1:
Tamanho do Espaço Virtual = 2^32 bytes = **4 GiB**.

Passo 2:
Tamanho da Página = 4 KiB = 2^12 bytes.

Passo 3:
Páginas Virtuais = 2^32 / 2^12 = 2^20 páginas (**1.048.576 entradas**).

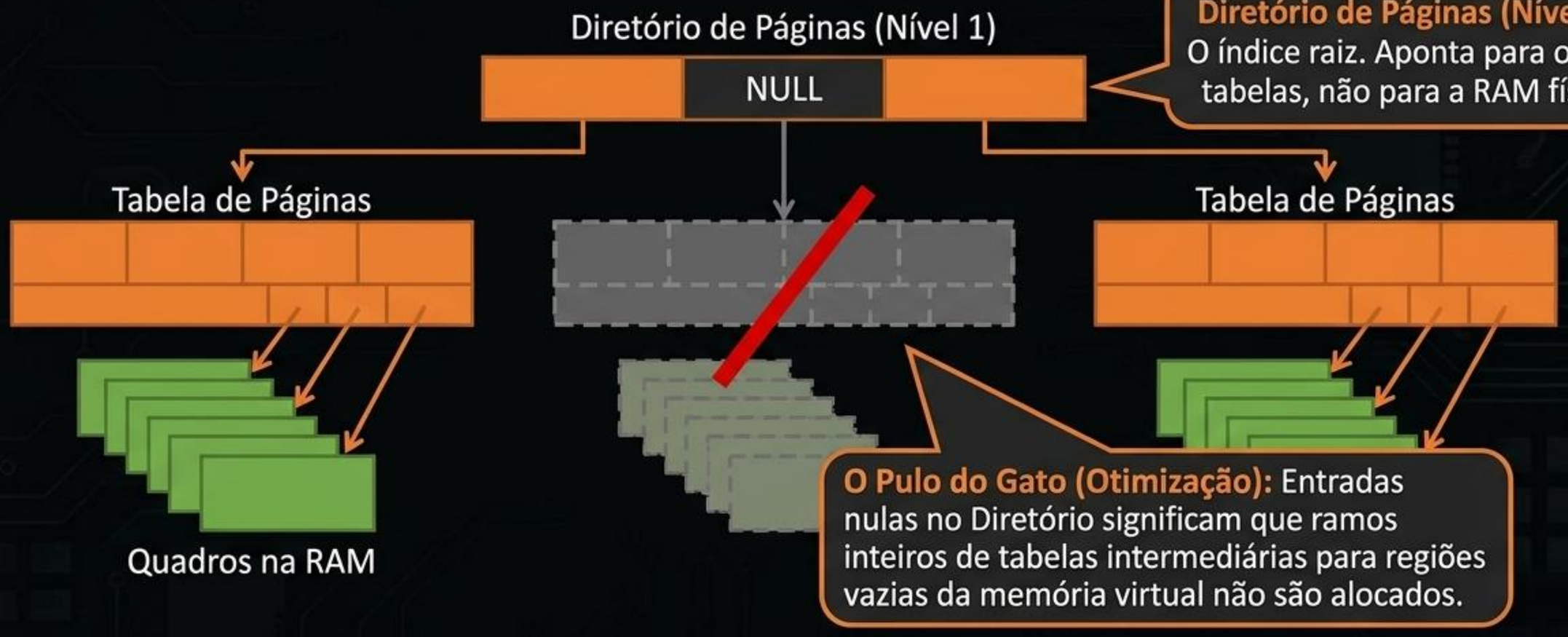
Passo 4:
Custo da Tabela Linear = 2^20 entradas × 4 bytes = 4.194.304 bytes
= **4 MiB** por processo!

O Impacto: Se tivermos 100 processos ativos, perderemos 400 MiB de RAM apenas para armazenar tabelas de mapeamento. É necessário um modelo dinâmico.

Visualização do Desperdício de Memória RAM

Gemini Notebook

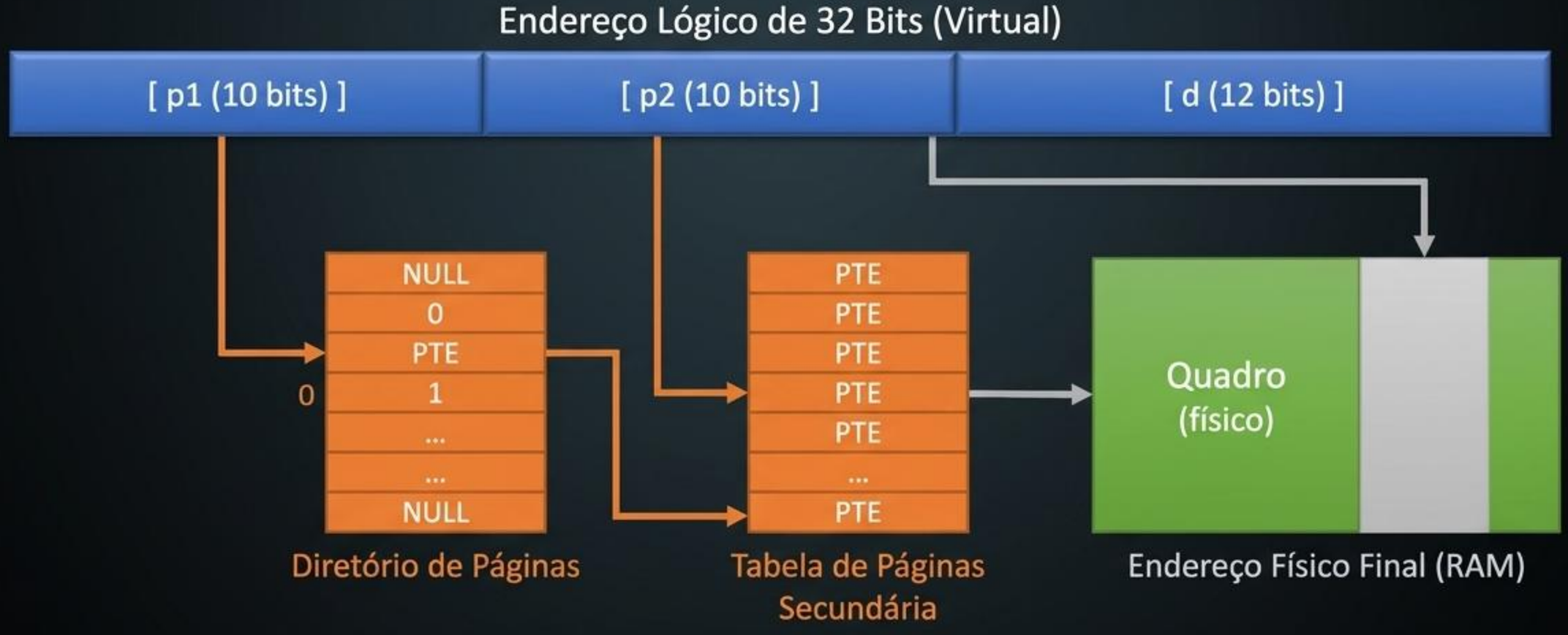
Hierarquia de Paginação: A Solução para o Desperdício (O Modelo Multinível)



Resultado: O overhead de memória encolhe drasticamente para processos esparsos.

Gemini Notebook

A Mecânica da Paginação em Dois Níveis: O Pulo do Gato



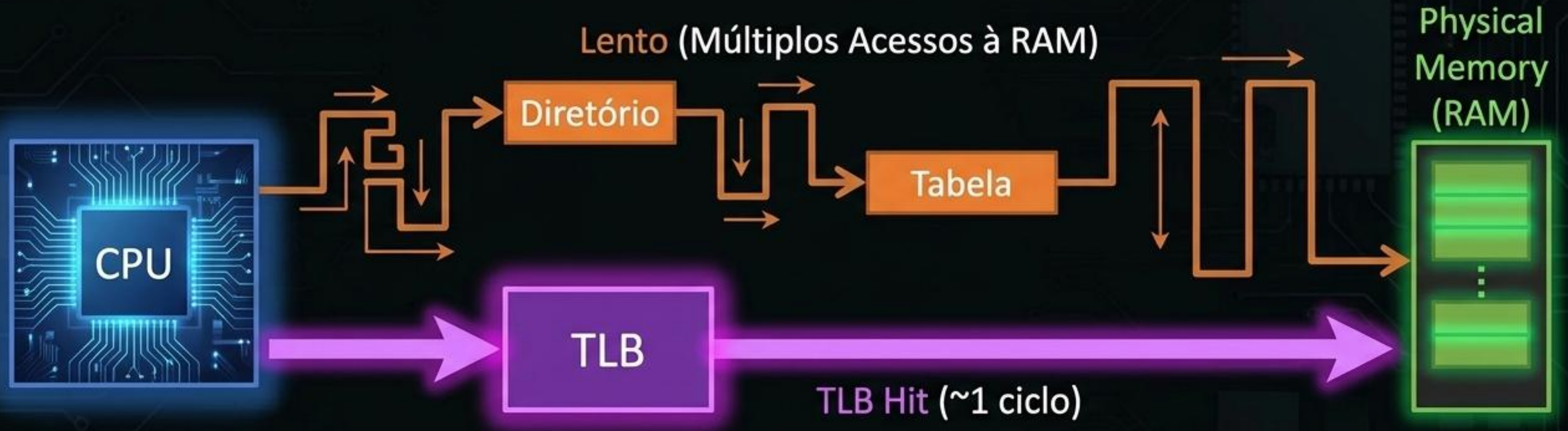
1. A MMU usa p1 (Índice Externo) para localizar a tabela secundária correta.
2. A MMU usa p2 (Índice Interno) para achar a PTE final contendo o número do quadro físico.
3. O quadro físico é combinado com o deslocamento inalterado d para acessar o byte exato na RAM.

Arquiteturas de 64 bits (como Intel Core i7) podem expandir isso para até 4 níveis (fatiando 9 bits por nível).

Gemini Notebook

Aceleração via TLB: O Caminho Rápido para a Memória

Contrastando o Page Table Walk (Lento) com o Bypass do TLB (Rápido)



O Custo do Multinível: Mais níveis poupam memória, mas exigem 2, 3 ou 4 leituras na RAM apenas para traduzir o endereço (**Page Table Walk**).

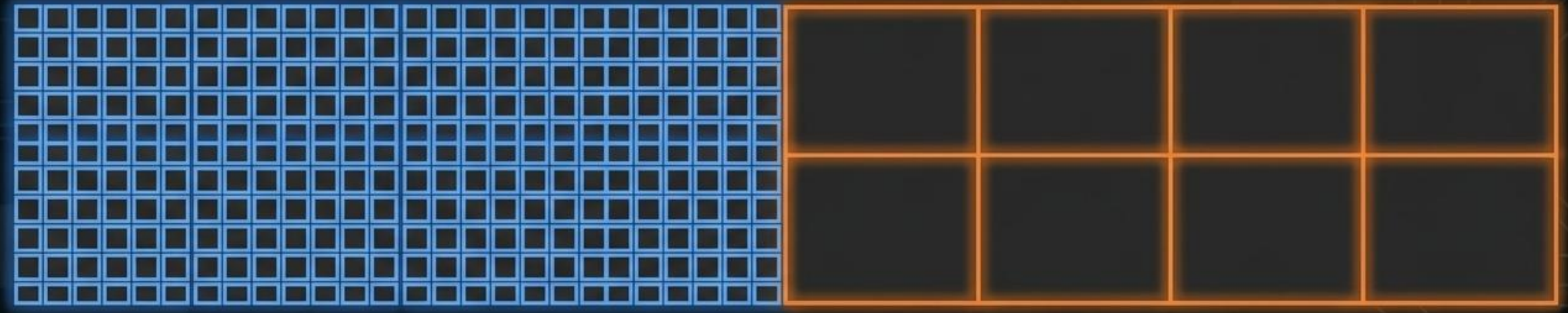
A Salvação (TLB): O *Translation Lookaside Buffer* protege o desempenho. Se o endereço estiver no cache (**Hit**), a penalidade do multinível desaparece. Em caso de **Miss**, o hardware ou SO sofre a punição de percorrer as tabelas.

Gemini Notebook

Trade-offs da Paginação: O Tamanho da Página Importa

Páginas Pequenas (ex: 4 KiB)

Páginas Grandes (ex: 2 MiB / 4 MiB)



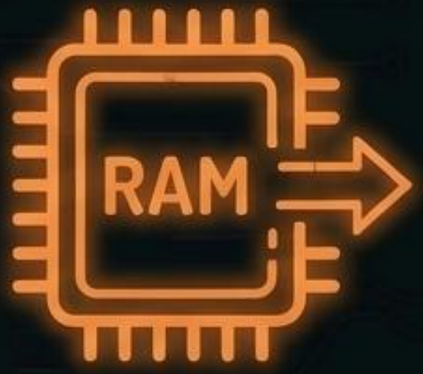
- (+) Menor fragmentação interna (menos desperdício por processo).
- (+) Granularidade fina para proteção (R/W isolado com precisão).
- (-) Exige tabelas de páginas muito maiores.
- (-) Menor Alcance da TLB, resultando em mais TLB misses.

- (+) Tabelas enxutas, reduzindo drasticamente o *overhead* de memória.
- (+) Maximiza o Alcance da TLB (uma entrada cobre muita memória).
- (-) Maior fragmentação interna.
- (-) Ineficiente para operações de E/S de pequenos blocos.

Gemini Notebook

Overhead da Paginação: Memória vs. Tempo (Trade-offs)

Overhead de Memória





Origem: Tabelas de páginas mapeando o espaço de endereçamento.

Soluções de Engenharia: Paginação Multinível (Alocação sob demanda); Tabelas Invertidas (Mapeamento pelo quadro físico em vez da página lógica); Páginas Maiores.

Overhead de Tempo



Origem: Buscas na RAM (Page Walks) e Context Switches que esvaziam o cache (TLB Flush).

Soluções de Engenharia: Caches TLB maiores/multinível; TLBs com ASID (Address Space ID) para evitar flushes na troca de contexto.

Gemini Notebook

Sem Fragmentação Externa: A paginação permite que o SO aloque quadros espalhados pela RAM. O processo enxerga tudo como contíguo.

O Custo - Fragmentação Interna: O tamanho da página é fixo. Se um processo precisa de 5 KiB e usamos páginas de 4 KiB, o SO deve alocar 2 quadros (8 KiB totais). Os 3 KiB restantes no último quadro estão alocados, mas inúteis.



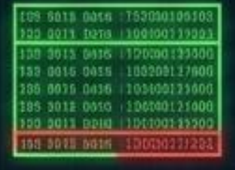
Regra Geral: O desperdício médio teórico é de 1/2 página por segmento de memória alocado.

Gemini Notebook

Contexto: Um processo requisita exatamente 74.000 bytes de memória.

Tamanho de Página = 4 KiB (4.096 bytes)

>_ Páginas necessárias:
74.000 ÷ 4.096 = 18,06
-> Arredonda para 19 páginas.

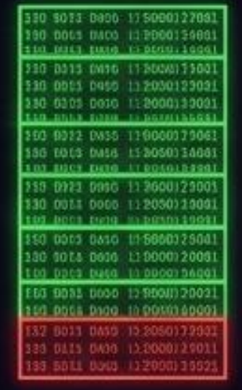


> Memória Alocada na RAM:
19 × 4.096 = 77.824 bytes.

> Desperdício (Fragmentação Interna):
77.824 - 74.000 = 3.824 bytes.
>

Tamanho de Página = 8 KiB (8.192 bytes)

>_ Páginas necessárias:
74.000 ÷ 8.192 = 9,03
-> Arredonda para 10 páginas.



>_ Memória Alocada na RAM:
10 × 8.192 = 81.920 bytes.

> Desperdício (Fragmentação Interna):
81.920 - 74.000 = 7.920 bytes.
>

Conclusão: Páginas maiores reduzem as tabelas e ajudam a TLB, mas dobram a fragmentação interna na borda dos segmentos.

Gemini Notebook

As 5 Leis do Gerenciamento de Memória

1. Paginação Linear tem $f = \frac{x + r^2}{n}$ tradução matemática simples, mas o *overhead* de memória inviabiliza seu uso moderno.

2. Tabelas Multinível salvam memória ao omitir regiões vazias, mas punem a velocidade com o temido *Page*





Endereço Virtual → Consulta TLB → Tabela Multinível → Quadro RAM

3. A TLB em hardware é a guardiã do desempenho, ocultando a latência estrutural do multinível.



4. Páginas Pequenas minimizam a fragmentação interna, mas inflacionam tabelas e sufocam a TLB.





5. Páginas Grandes comprimem tabelas e maximizam a TLB, ao custo de maior desperdício físico de RAM.



Gemini Notebook

Situação: Se um sistema de 64 bits tentar usar uma tabela linear com páginas de 4 KiB, o que acontece?

Diagnóstico: A tabela precisaria de 2^52 entradas, consumindo petabytes de memória. Seria fisicamente impossível armazenar a tabela de um único processo na RAM. **Solução:** 4 ou mais níveis de paginação.

Situação: Por que o sistema inteiro pode sofrer uma leve queda temporária de desempenho logo após uma mudança de contexto (Context Switch) entre processos diferentes?

Diagnóstico: Ocorre o TLB Flush. A tabela de páginas antiga é desativada. A TLB é esvaziada. O novo processo começará a rodar sofrendo uma rajada de TLB Misses até que a TLB seja aquecida com as novas traduções.

Gemini Notebook



“A arte de projetar Sistemas Operacionais não reside em buscar soluções livres de defeitos, mas na escolha milimétrica dos trade-offs entre Tempo e Espaço.”

Bibliografia e Fontes Base:

- SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. Fundamentos de Sistemas Operacionais. 9ª ed. LTC. (Foco em Cap. 8 e 9).
- MAZIERO, C. Sistemas Operacionais: Conceitos e Mecanismos. (Foco em Paginação e TLB).
- TANENBAUM, A. S. Sistemas Operacionais Modernos.

Material desenvolvido para o curso de Bacharelado em Ciência da Computação, UNEMAT – Campus de Alto Araguaia.
Estruturação baseada nas orientações do Prof. Me. Juvenal Neto.

Gemini Notebook